

Mini Regression Project(python)

Proble: House Price Preddiction

```
#to see what datasets are there
import sklearn.datasets
print(dir(sklearn.datasets))

['_all_', '__builtins__', '__cached__', '__doc__', '__file__',
 '__getattr__', '__loader__', '__name__', '__package__', '__path__',
 '__spec__', '_arff_parser', '_base', '_california_housing',
 '_covtype', '_kddcup99', '_lfw', '_olivetti_faces', '_openml',
 '_rcv1', '_samples_generator', '_species_distributions',
 '_svmlight_format_fast', '_svmlight_format_io', '_twenty_newsgroups',
 'clear_data_home', 'dump_svmlight_file', 'fetch_20newsgroups',
 'fetch_20newsgroups_vectorized', 'fetch_california_housing',
 'fetch_covtype', 'fetch_file', 'fetch_kddcup99', 'fetch_lfw_pairs',
 'fetch_lfw_people', 'fetch_olivetti_faces', 'fetch_openml',
 'fetch_rcv1', 'fetch_species_distributions', 'get_data_home',
 'load_breast_cancer', 'load_diabetes', 'load_digits', 'load_files',
 'load_iris', 'load_linnerud', 'load_sample_image',
 'load_sample_images', 'load_svmlight_file', 'load_svmlight_files',
 'load_wine', 'make_biclusters', 'make_blobs', 'make_checkerboard',
 'make_circles', 'make_classification', 'make_friedman1',
 'make_friedman2', 'make_friedman3', 'make_gaussian_quantiles',
 'make_hastie_10_2', 'make_low_rank_matrix', 'make_moons',
 'make_multilabel_classification', 'make_regression', 'make_s_curve',
 'make_sparse_coded_signal', 'make_sparse_spd_matrix',
 'make_sparse_uncorrelated', 'make_spd_matrix', 'make_swiss_roll',
 'textwrap']

##Step 1: Import libraries
import numpy as np
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error,
r2_score
#Step 2: Load dataset
data = fetch_california_housing()
X = data.data
y = data.target
#Step 3: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
#Step 4: Train model
model = LinearRegression()
```

```

model.fit(X_train, y_train)
#Step 5: Predict
y_pred = model.predict(X_test)
#Step 6: Evaluate metrics
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("MSE:", mse)
print("RMSE:", rmse)
print("MAE:", mae)
print("R2 Score:", r2)
#Step 7: Adjusted R2
n = X_test.shape[0]
p = X_test.shape[1]
adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
print("Adjusted R2:", adjusted_r2)

MSE: 0.5558915986952425
RMSE: 0.7455813830127751
MAE: 0.5332001304956989
R2 Score: 0.5757877060324521
Adjusted R2: 0.5749637928613571

```

Classification

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
# Load dataset
df = pd.read_csv("student_data.csv")
# Features (X) and Target (y)
X = df[['Hours_Studied']]
y = df['Pass']
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
# Create model
model = LogisticRegression()
# Train model
model.fit(X_train, y_train)
# Predictions
y_pred = model.predict(X_test)
# Predict new student
new_student = pd.DataFrame({
    'Hours_Studied': [6]
})

```

```
})  
prediction = model.predict(new_student)  
print(y_pred)  
print("Prediction (0=Fail, 1=Pass):", prediction[0])
```

```
[1 0]  
Prediction (0=Fail, 1=Pass): 1
```